



# Distributed Query Processing



# OUTLINE

---

- ✓ What is Query ?
- ✓ What is Query Processor?
- ✓ Main Function of a Query Processor
- ✓ Main Problems of Query Processing
- ✓ Characteristics of Query Processor
- ✓ Main layers of Query Processing
- ✓ Query Decomposition
- ✓ Data Localization
- ✓ Global Query Optimization
- ✓ Distributed Execution



# Distributed Query Processing

---

## ❑ What is Query ?

- A **query** is a statement requesting the retrieval of information.
- The portion of a DML that involves information retrieval is called a **query language**.



# Distributed Query Processing

---

## ❑ What is Query Processor?

- In relational database technology, users perform the task of data processing and data manipulation with the help of high-level non-procedural language (e.g. SQL).
- This **high-level query hides the low-level details from the user** about the physical organization of the data and presents such an environment so that the user can handle the tasks of even complex queries in an easy, concise and simple fashion.
- **The inner procedure of query-task is performed by a DBMS module called a Query Processor.** Hence the users are relieved from query optimization which is a time-consuming task that is actually performed by the query processor.



# Distributed Query Processing

---

## ❑ What is Query Processor?

- Understanding Query processing in distributed database environments is very difficult instead of centralized database, because there are many elements or parameters involved that affect the overall performance of distributed queries. Moreover, in distributed environment, the query processor may have to access in many sites. Thus, query response time may become very high.
- That is why, Query processing problem is divided into several sub-problems/ steps which are easier to solve individually.



# Distributed Query Processing

---

## ❑ Main Function/Objective of a Query Processor

- Main function of a query processor is to **transform a high-level-query (also called calculus query) into an equivalent lower-level query (also called algebraic query).**
- **The conversion must be correct and efficient. The conversion will be correct if low-level query has the same semantics as the original query, i.e. if both queries produce the same result.**



# Distributed Query Processing

## ❑ Main Problems of Query Processing

- Main problem of query processing is **query optimization**.
- It is a **time consuming task**, because many **execution strategies are involved to minimize (optimize) computer resource consumption**.
- Total cost also affects query processing. Cost depends on the time required for processing operations of the query at various sites, utilization of various computer resources etc, time needed for exchanging data between sites participating in the execution of the query etc.
- **Time and space required to process the query is also an important factor** for the performance of the query processing.





# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor

- Languages
- Types of Optimization
- Optimization Timing
- Statistics
- Decision Sites
- Exploitation of the Network Topology
- Exploitation of Replicated Fragments
- Use of Semi-join





# Distributed Query Processing

## ❑ Important Characteristics of Query Processor

### Languages:

- Input language to the query processor is based on high-level relational DBMS, i.e. **input is in calculus query**.
- On the other hand, output language is based on lower-level relational DMBS, i.e. **output is in algebraic query**.
- The operations of the output language are implemented directly in the computer system.
- Query processing must perform efficient mapping from the input language to the output language.



# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor

### Types of Optimization:

- Among all possible strategies for executing query, the one in which **less time and space** are required is the best solution for the optimization of query.
- A comparative study for all the strategies should perform such that how much cost will be required for each solution. That strategy should be chosen in which there will be less cost. But keep in mind that choosing the low-cost strategy may require much space. So care should be taken in this case also.



# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor

### Optimization Timing:

- The actual time required to optimize the execution of a query is an important factor. **If less time is required, then it is the best solution for query processing.**



# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor Statistics:

- The effectiveness of query optimization relies on statistical information of the database, i.e. **how many fragments** query will be needed, **which operation** should be done first, **size and number of distinct values** of each attribute of the relation, histograms of the attributes for minimizing probability error etc.



# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor

### **Decision Sites:**

- In a distributed database, several sites may participate for answering the query. Most systems use centralized decision approach, in which a single site generates the strategy. However, the decision process could be distributed among various sites.

### **Exploitation of Network Topology:**

- Distributed query processor uses computer network, so its performance depends also on which topology it is using. The cost function, speed, utilization of various network resources are important factors for executing query processor in a distributed environment.



# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor

### Exploitation of Replicated Fragments:

- A distributed relation is usually divided into relation fragments.
- Distributed queries expressed on global relations are mapped into queries on physical fragments of relations by translating relations into fragments. We call this process localization because its main function is to localize the data involved in the query.
- For higher reliability and better read performance, it is useful to have fragments replicated at different sites.





# Distributed Query Processing

---

## ❑ Important Characteristics of Query Processor

### Use of Semi-join operation:

- In distributed system, it is better to use semi-join operation instead of join operation **for minimizing data communication.**





# Distributed Query Processing

---

## □ Layers of Query Processing

- Processing of Query in distributed DBMS instead of centralized (Local) DBMS.
- Understanding Query processing in distributed database environments is very difficult instead of centralized database, because there are many elements involved.
- So, Query processing problem is divided into several sub-problems/ steps which are easier to solve individually.
- A general layering scheme for describing distributed query processing is given below:



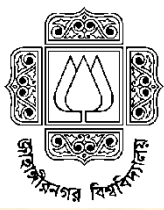
# Distributed Query Processing

---

## ❑ Main layers of Query Processing

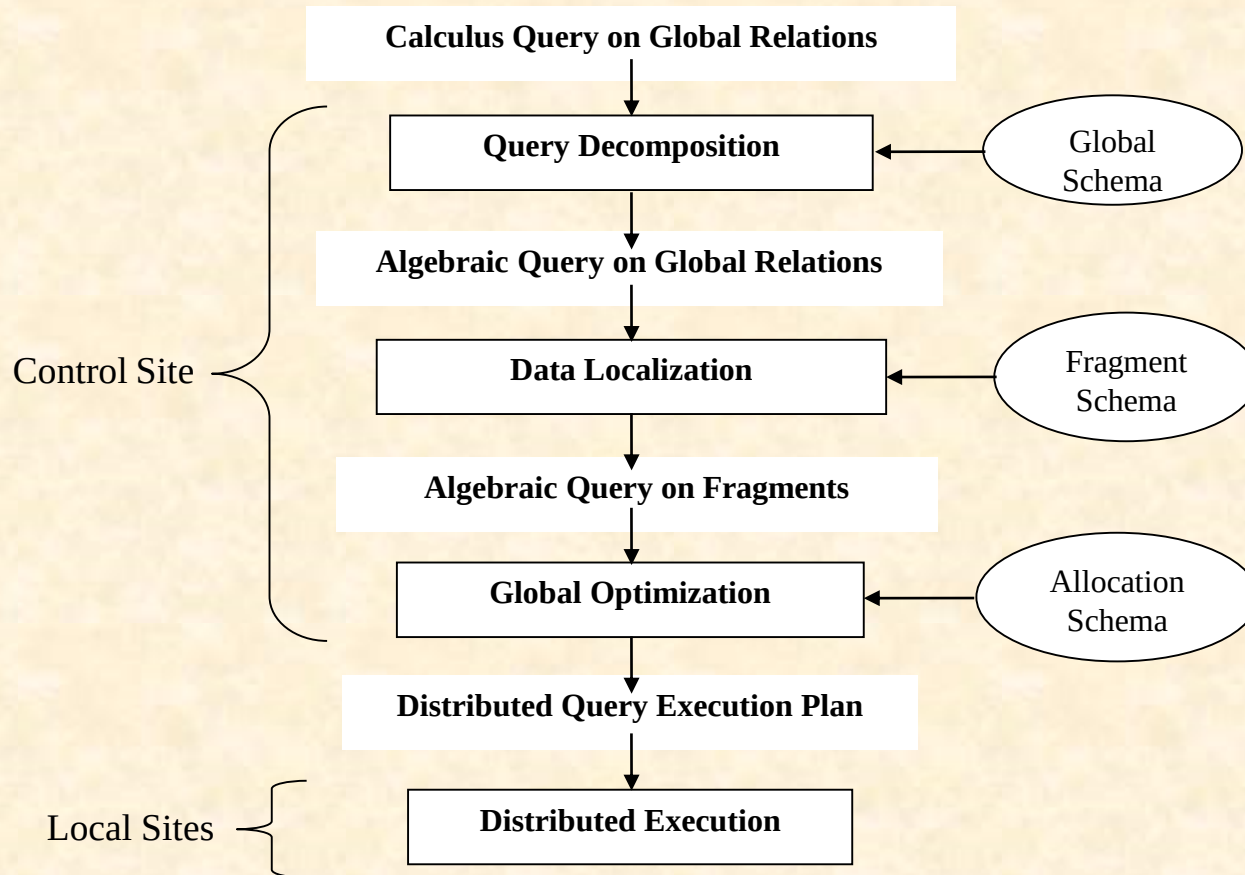
Query processing involves 4 main layers:

- Query Decomposition
- Data Localization
- Global Query Optimization
- Distributed Execution



# Distributed Query Processing

## □ Main layers of Query Processing



**Fig. Generic Layering Scheme for Distributed Query Processing**



# Distributed Query Processing

---

## ❑ Main layers of Query Processing

- The input is a query on global data expressed in relational calculus. This query is posed on global (distributed) relations, meaning that data distribution is hidden.
- The first three layers map the input query into an optimized distributed query execution plan. They perform the functions of query decomposition, data localization, and global query optimization.
- Query decomposition and data localization correspond to query rewriting.
- The first three layers are performed by a central control site and use schema information stored in the global directory.
- The fourth layer performs distributed query execution by executing the plan and returns the answer to the query. It is done by the local sites and the control site.



- The first layer decomposes the calculus query into an algebraic query on global relations. The information needed for this transformation is found in the global conceptual schema describing the global relations.
- Both input and output queries refer to global relations, without knowledge of the distribution of data. Therefore, query decomposition is the same for centralized and distributed systems.
- Query decomposition can be viewed as four successive steps:
  - **1) Normalization,**
  - 2) Analysis,**
  - 3) Elimination of redundancy, and**
  - 4) Rewriting.**



# Distributed Query Processing

---

## ❑ Query Decomposition

- First, the calculus query is rewritten in a normalized form that is suitable for subsequent manipulation. Normalization of a query generally involves the manipulation of the query quantifiers and of the query qualification by applying logical operator priority.
- Second, the normalized query is analyzed semantically so that incorrect queries are detected and rejected as early as possible. Typically, they use some sort of graph that captures the semantics of the query.





# Distributed Query Processing

---

## □ Query Decomposition

- Third, the correct query is simplified. One way to simplify a query is to eliminate redundant predicates.
- Fourth, the calculus query is restructured as an algebraic query. Several algebraic queries can be derived from the same calculus query, and that some algebraic queries are “better” than others. The quality of an algebraic query is defined in terms of expected performance.





# Distributed Query Processing

## □ Normalization

- The input query may be arbitrarily complex.
- It is the goal of normalization to transform the query to a normalized form to facilitate further processing.
- With relational languages such as SQL, the most important transformation is that of the query qualification (the WHERE clause), which may be an arbitrarily complex, quantifier-free predicate, preceded by all necessary quantifiers ( $\forall$  or  $\exists$ ).
- There are two possible normal forms for the predicate, one giving precedence to the AND ( $\wedge$ ) and the other to the OR ( $\vee$ ). The **conjunctive normal form** is a conjunction ( $\wedge$  predicate) of disjunctions ( $\vee$  predicates) as follows:

$$(p_{11} \vee p_{12} \vee \dots \vee p_{1n}) \wedge \dots \wedge (p_{m1} \vee p_{m2} \vee \dots \vee p_{mn})$$

where  $p_{ij}$  is a simple predicate.



# Distributed Query Processing

## □ Normalization

- A qualification in **disjunctive normal form**, on the other hand, is as follows:

$$(\mathbf{p_{11} \wedge p_{12} \wedge \dots \wedge p_{1n}}) \vee \dots \vee (\mathbf{p_{m1} \wedge p_{m2} \wedge \dots \wedge p_{mn}})$$

- The transformation is straightforward using the well-known equivalence rules for logical operations ( $\wedge$ ,  $\vee$  and  $\neg$ ):

1.  $p_1 \wedge p_2 \iff p_2 \wedge p_1$
2.  $p_1 \vee p_2 \iff p_2 \vee p_1$
3.  $p_1 \wedge (p_2 \wedge p_3) \iff (p_1 \wedge p_2) \wedge p_3$
4.  $p_1 \vee (p_2 \vee p_3) \iff (p_1 \vee p_2) \vee p_3$
5.  $p_1 \wedge (p_2 \vee p_3) \iff (p_1 \wedge p_2) \vee (p_1 \wedge p_3)$
6.  $p_1 \vee (p_2 \wedge p_3) \iff (p_1 \vee p_2) \wedge (p_1 \vee p_3)$
7.  $\neg (p_1 \wedge p_2) \iff \neg p_1 \vee \neg p_2$
8.  $\neg (p_1 \vee p_2) \iff \neg p_1 \wedge \neg p_2$
9.  $\neg (\neg p) \iff p$



# Distributed Query Processing

---

- ❑ **Example:** Let us consider the following query on the engineering database that we have been referring to:

“Find the names of employees who have been working on project P1 for 12 or 24 months”

## Engineering Database:

EMP(ENO, ENAME, TITLE)

PROJ(PNO, PNAME, BUDGET).

SAL(TITLE, AMT)

ASG(ENO, PNO, RESP, DUR)

;SAL=Salary, AMT=Amount

; Employees assigned to projects

; RESP=Responsibility, DUR=Duration



# Distributed Query Processing

---

- ❑ **Example:** Let us consider the following query on the engineering database that we have been referring to:

“Find the names of employees who have been working on project P1 for 12 or 24 months”

The query expressed in SQL is

```
SELECT  ENAME
FROM    EMP, ASG
WHERE   EMP.ENO = ASG.ENO
        AND ASG.PNO = "P1"
        AND DUR = 12 OR DUR = 24
```



# Distributed Query Processing

❑ **Example:** Let us consider the following query on the engineering database that we have been referring to:

“Find the names of employees who have been working on project P1 for 12 or 24 months”

The qualification in **conjunctive normal form** is

$$\text{EMP.ENO} = \text{ASG.ENO} \wedge \text{ASG.PNO} = \text{“P1”} \wedge (\text{DUR} = 12 \vee \text{DUR} = 24)$$

The qualification in **disjunctive normal form** is

$$(\text{EMP.ENO} = \text{ASG.ENO} \wedge \text{ASG.PNO} = \text{“P1”} \wedge \text{DUR} = 12) \vee (\text{EMP.ENO} = \text{ASG.ENO} \wedge \text{ASG.PNO} = \text{“P1”} \wedge \text{DUR} = 24)$$



# Distributed Query Processing

---

## □ Analysis

- Query analysis enables rejection of normalized queries for which further processing is either impossible or unnecessary.
- The main reasons for rejection are that the query is **type incorrect** or **semantically incorrect**.
- A query is type incorrect if any of its attribute or relation names are not defined in the global schema, or if operations are being applied to attributes of the wrong type.
- The technique used to detect type incorrect queries is similar to type checking for programming languages.





# Distributed Query Processing

---

## ❑ Analysis

**Example:** The following SQL query on the engineering database is type incorrect for two reasons. First, **attribute E# is not declared** in the schema. Second, the operation “>200” is **incompatible with the type string of ENAME**.

```
SELECT E#  
FROM EMP  
WHERE ENAME > 200
```



# Distributed Query Processing

---

## □ Analysis

- A query is semantically incorrect if **its components do not contribute in any way to the generation of the result.**
- This is based on the representation of the query as a graph, called a **query graph** or **connection graph**.
- In a query graph, one node indicates the result relation, and any other node indicates an operand relation. An edge between two nodes one of which does not correspond to the result represents a join, whereas an edge whose destination node is the result represents a project.
- An important subgraph of the query graph is the join graph, in which only the joins are considered.



# Distributed Query Processing

## □ Analysis

**Example:** Let us consider the following query:

“Find the names and responsibilities of programmers who have been working on the CAD/CAM project for more than 3 years.”

The query expressed in SQL is

```
SELECT      ENAME, RESP
FROM        EMP, ASG, PROJ
WHERE       EMP.ENO = ASG.ENO
            AND ASG.PNO = PROJ.PNO
            AND PNAME = "CAD/CAM"
            AND DUR ≥ 36
            AND TITLE = "Programmer"
```



# Distributed Query Processing

## □ Analysis

Query Graph:

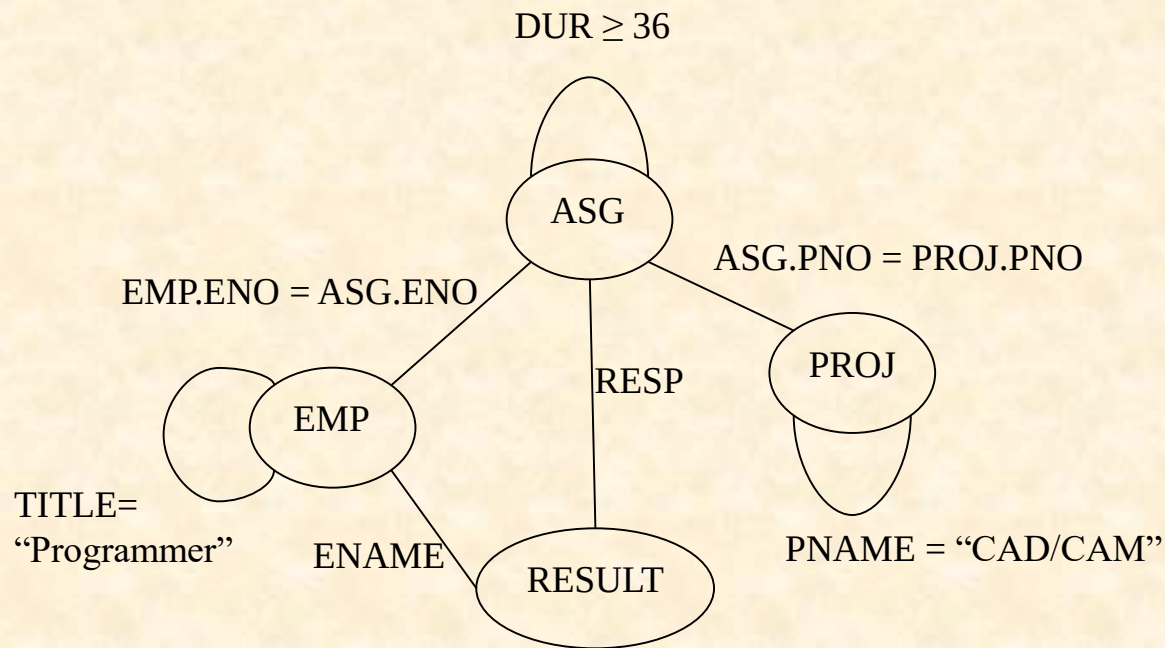


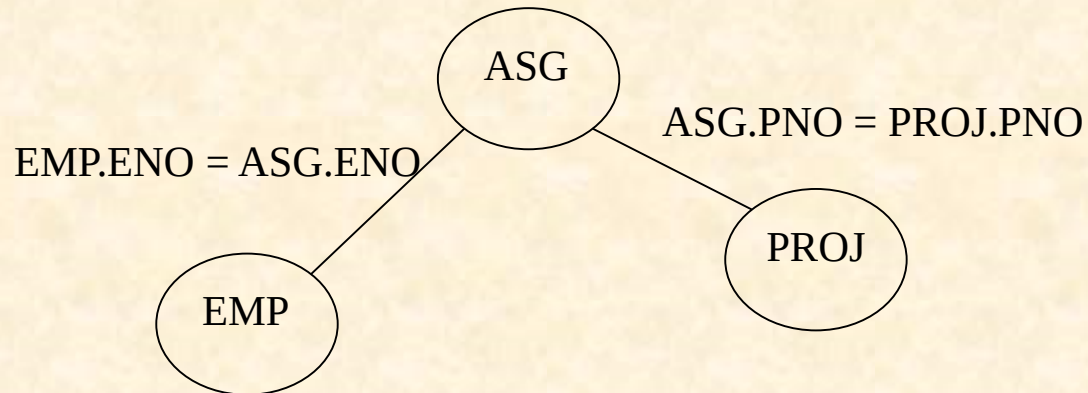
Fig.: Query Graph



# Distributed Query Processing

## □ Analysis

Join Graph:



**Fig.: Corresponding Join Graph**



# Distributed Query Processing

---

## □ Analysis

A query is semantically incorrect if its query graph is not connected.

**Example:** Let us consider the following SQL query:

```
SELECT      ENAME, RESP
FROM        EMP, ASG, PROJ
WHERE       EMP.ENO = ASG.ENO
           AND PNAME = "CAD/CAM"
           AND DUR_ 36
           AND TITLE = "Programmer"
```





# Distributed Query Processing

## ❑ Analysis

It's query graph, shown below is disconnected, which tells us that the query is semantically incorrect.

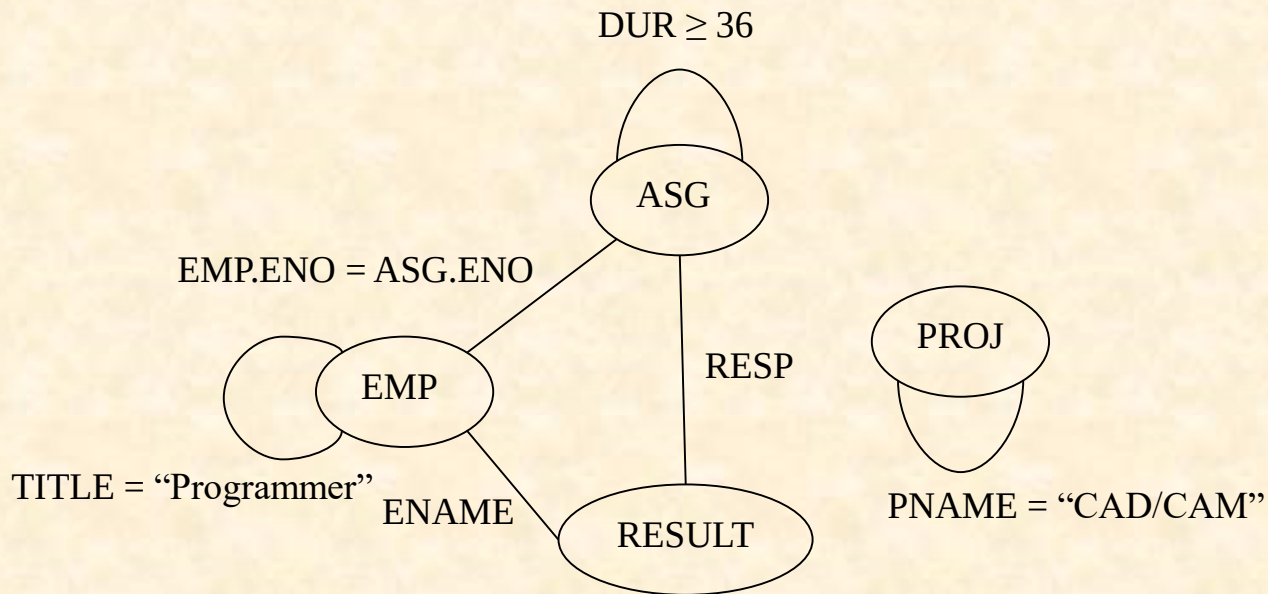


Fig.: Query Graph



# Distributed Query Processing

## ❑ Elimination of Redundancy

- Simplify the query by eliminating redundancies, e.g., redundant predicates.
- Redundancies are often due to semantic integrity constraints expressed in the query language.
- Transformation rules are used:

$$P \wedge P \Leftrightarrow P$$

$$P \vee P \Leftrightarrow P$$

$$P \wedge \text{true} \Leftrightarrow P$$

$$P \vee \text{false} \Leftrightarrow P$$

$$P \wedge \text{false} \Leftrightarrow \text{false}$$

$$P \vee \text{true} \Leftrightarrow \text{true}$$

$$P \vee \neg P \Leftrightarrow \text{true}$$

$$P \wedge \neg P \Leftrightarrow \text{false}$$

$$P1 \wedge (P1 \vee P2) \Leftrightarrow P1$$

$$P1 \vee (P1 \wedge P2) \Leftrightarrow P1$$



# Distributed Query Processing

---

## ❑ Elimination of Redundancy

**Example:**

```
SELECT  TITLE
FROM    EMP
WHERE   (NOT (TITLE = "Programmer")
        AND (TITLE = "Programmer"
        OR  TITLE = "Elect. Eng.")
        AND NOT (TITLE = "Elect. Eng.))
        OR  ENAME = "Mr. X"
```



# Distributed Query Processing

## ❑ Elimination of Redundancy

### Example:

Can be simplified using the previous rules to become

```
SELECT  TITLE
FROM    EMP
WHERE   ENAME = " Mr. X"
```

The simplification proceeds as follows:

Let  $p_1$  be  $TITLE = \text{"Programmer"}$ ,  
 $p_2$  be  $TITLE = \text{"Elect. Eng."}$ , and  
 $p_3$  be  $ENAME = \text{"Mr. X"}$ .

The query qualification is:

$$(\neg p_1 \wedge (p_1 \vee p_2) \wedge \neg p_2) \vee p_3$$



# Distributed Query Processing

## ❑ Elimination of Redundancy

The disjunctive normal form for this qualification is obtained by applying rule 5 which yields:

$$(\neg p_1 \wedge ((p_1 \wedge \neg p_2) \vee (p_2 \wedge \neg p_2))) \vee p_3$$

and then rule 3 yields:

$$(\neg p_1 \wedge p_1 \wedge \neg p_2) \vee (\neg p_1 \wedge p_2 \wedge \neg p_2) \vee p_3$$

By applying rule 7, we obtain

$$(\text{false} \wedge \neg p_2) \vee (\neg p_1 \wedge \text{false}) \vee p_3$$

By applying the same rule, we get

$$(\text{false} \vee \text{false}) \vee p_3$$

**which is equivalent to  $p_3$  by rule 4.**



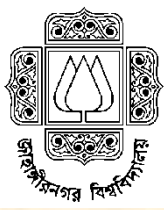
# Distributed Query Processing

---

## □ Rewriting

- The last step of query decomposition rewrites the query in **relational algebra**.
- For the sake of clarity it is customary to represent the relational algebra query graphically by an **operator tree**.
- An operator tree is a tree in which a leaf node is a relation stored in the database, and a non-leaf node is an intermediate relation produced by a relational algebra operator. The sequence of operations is directed from the leaves to the root, which represents the answer to the query.





# Distributed Query Processing

---

## □ Rewriting

The transformation of a tuple relational calculus query into an operator tree can easily be achieved as follows:

- First, a different leaf is created for each different tuple variable (corresponding to a relation). In SQL, the leaves are immediately available in the **FROM clause**.
- Second, **the root node is created as a project operation** involving the result attributes. These are found in the **SELECT clause in SQL**.
- Third, the qualification (SQL **WHERE clause**) is translated into the appropriate sequence of relational operations (select, join, union, etc.) going from the leaves to the root. The sequence can be given directly by the order of appearance of the predicates and operators.



# Distributed Query Processing

---

## ❑ Rewriting

### Example:

“Find the names of employees other than Mr. X who worked on the CAD/CAM project for either one or two years” whose SQL expression is:

```
SELECT  ENAME
FROM    PROJ, ASG, EMP
WHERE   ASG.ENO = EMP.ENO
        AND  ASG.PNO = PROJ.PNO
        AND  ENAME != "Mr. X"
        AND  PROJ.PNAME = "CAD/CAM"
        AND  (DUR = 12 OR DUR = 24)
```



# Distributed Query Processing

## □ Rewriting

Operator Tree:

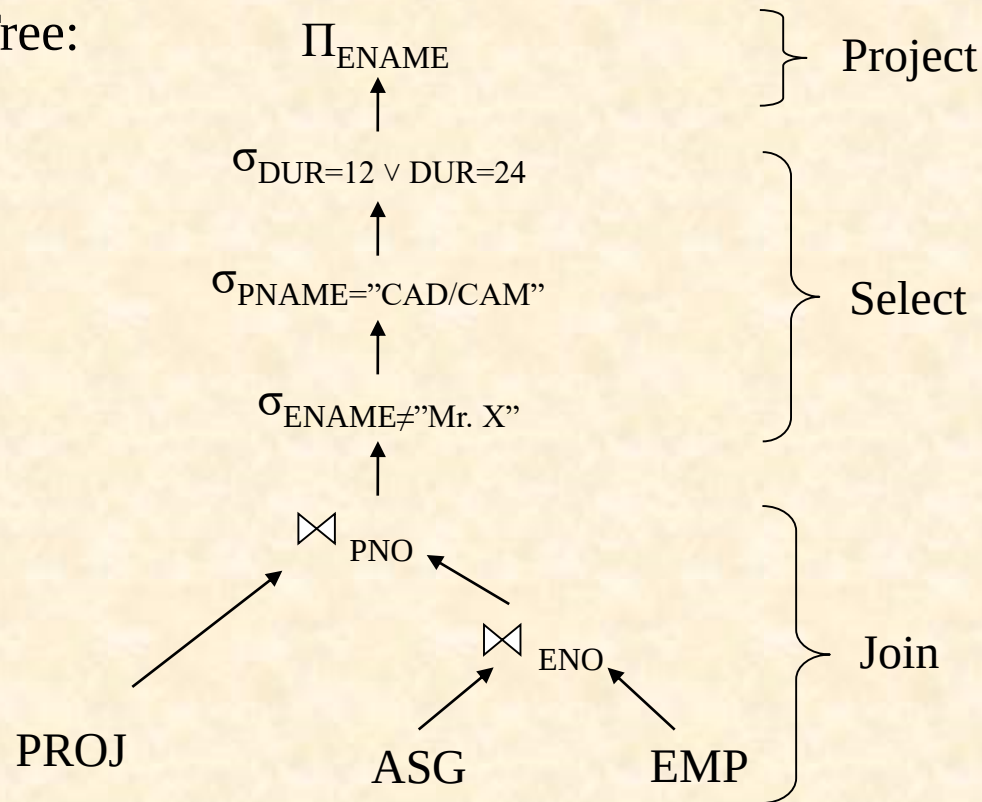


Fig: Operator tree



# Distributed Query Processing

---

## ❑ Localization of Distributed Data

- Output of the first layer is an **algebraic query** on distributed relations which is input to the second layer.
- The main role of this layer is to **localize the query's data** using data distribution information.
- We know that relations are fragmented and stored in disjoint subsets, called fragments where each fragment is stored at different site.



# Distributed Query Processing

---

## □ Localization of Distributed Data

- This layer **determines which fragments are involved in the query and transforms the distributed query into a fragment query.**
- A naive way to localize a distributed query is to generate a query where each global relation is substituted by its localization program. This can be viewed as replacing the leaves of the operator tree of the distributed query with subtrees corresponding to the localization programs. We call the query obtained this way the localized query.



# Distributed Query Processing

---

## ❑ Global Query Optimization

- The input to the third layer is a **fragment algebraic query**.
- The goal of this layer is to **find an execution strategy** for the algebraic fragment query which is **close to optimal**.
- Query optimization consists of
  - i) Finding the best **ordering of operations** in the fragment query,
  - ii) Finding the **communication operations** which minimize a cost function.



# Distributed Query Processing

---

## ❑ Global Query Optimization

- The cost function refers to computing resources such as **disk space, disk I/Os, buffer space, CPU cost, communication cost**, and so on.
- Query optimization is achieved through the **semi-join** operator instead of join operators.





# Distributed Query Processing

---

## ❑ Distributed Execution

- The last layer is performed by all the sites having fragments involved in the query.
- Each subquery, called a local query, is executing at one site. It is then optimized using the local schema of the site.